

Semester	B.E. Semester VII – Computer Engineering
Subject	R Programming
Subject Professor In- charge	Prof. Sanjeev Dwivedi
Assisting Teachers	Prof. Sanjeev Dwivedi
Laboratory	M-411B

Student Name	Sneha Gadhari
Roll Number	23102B0055
Grade and Subject Teacher's Signature	

Experiment Number	02
Experiment Title	Control Flow for Data Cleaning & Advanced Missing Data Handling

Resources / Apparatus Required	Hardware: Computer system	Software: Colab
--------------------------------------	---------------------------------	--------------------

Objective	To apply core R programming concepts - conditional logic, custom functions, loop-based vs. vectorized processing, exception handling with tryCatch(), and systematic missing-data detection and imputation - in order to clean, validate, and prepare two real-world datasets (UCI Heart Disease and UCI Adult/Census Income) for downstream statistical analysis.
Description	This assignment consists of two connected R programming laboratory exercises that together simulate the real-world data-cleaning workflow used by data analysts before any statistical modeling can begin. Both labs work with authentic, messy healthcare and demographic datasets, and both require students to first deliberately corrupt clean data - injecting negative values, missing values, extreme outliers, blank strings, and impossible entries - so that the detection and treatment logic can be tested against known, controlled problems rather than hoped-for ones. Lab 2a: Control Flow for Data Cleaning uses the UCI Heart Disease dataset, focusing on the trestbps (resting blood pressure) variable. It builds a complete cleaning pipeline for a single numeric variable and layers in performance analysis: • A custom if-else based function classifies each BP reading as negative (→ converted to NA), extreme (→ capped at 250 mmHg), or valid (→ left unchanged). • tryCatch() wraps two operations that can fail in practice: computing a mean BP when values are missing, and computing a chol / trestbps ratio where the denominator might be zero, NA, or otherwise invalid - both return informative messages instead of crashing the script. • The same outlier-detection logic is implemented twice - once with an explicit for loop, once with vectorized R operations - and their runtimes are benchmarked with system.time() and microbenchmark() to make the performance gap concrete rather than theoretical. • The cleaned column is validated (missing count, min/max/mean/median, and confirmation that no negative or >250 values remain) and exported as cleaned_heart_data.csv. Lab 2b: Advanced Missing Data Handling uses the UCI Adult/Census Income dataset and broadens the scope from one numeric variable to a full mixed-type dataset (numeric + categorical). Its emphasis is conceptual precision as much as

	mechanics: • It distinguishes four related but distinct ideas that are frequently confused: NA (a missing value inside a vector), NaN (an undefined numeric result, technically a subtype of NA), NULL (the absence of an R object entirely, which cannot literally occupy a data-frame cell), and "" (a blank string - a valid-looking value that is semantically missing). • Detection uses <code>is.na()</code>, <code>is.nan()</code>, <code>is.null()</code>, direct string comparison for blanks, a logical check for impossible values (<code>age == 999</code>), and <code>naniar::miss_var_summary()</code> for a variable-by-variable missingness report. • Treatment converts impossible values to NA, replaces blank/missing categorical entries with "Unknown", median-imputes numeric gaps (via a reusable custom function), and drops rows with unrecoverable NaN - with <code>complete.cases()</code> used to track fully complete vs. incomplete observations throughout. • Missingness is quantified and visualized (via <code>naniar::gg_miss_var()</code>) both before and after treatment, and the final dataset is validated with <code>skimr::skim()</code> before being exported as <code>cleaned_adult_data.csv</code>. Across both labs, the underlying skill being exercised is the same: recognizing that raw data cannot be trusted, building reusable and defensive functions to detect and fix specific classes of problems, and proving - through validation checks, timing benchmarks, and missingness reports - that the cleaning actually worked rather than just assuming it did.
Output	2a:

```
'data.frame': 303 obs. of 14 variables:
 $ age      : num  63 67 67 37 41 56 62 57 63 53 ...
 $ sex      : num  1 1 1 1 0 1 0 0 1 1 ...
 $ cp       : num  1 4 4 3 2 2 4 4 4 4 ...
 $ trestbps : num  145 160 120 130 130 120 140 120 130 140 ...
 $ chol     : num  233 286 229 250 204 236 268 354 254 203 ...
 $ fbs      : num  1 0 0 0 0 0 0 0 1 ...
 $ restecg  : num  2 2 2 0 2 0 2 0 2 2 ...
 $ thalach  : num  150 108 129 187 172 178 160 163 147 155 ...
 $ exang    : num  0 1 1 0 0 0 0 1 0 1 ...
 $ oldpeak  : num  2.3 1.5 2.6 3.5 1.4 0.8 3.6 0.6 1.4 3.1 ...
 $ slope    : num  3 2 2 3 1 1 3 1 2 3 ...
 $ ca       : num  0 3 2 0 0 0 2 0 1 0 ...
 $ thal     : num  6 3 7 3 3 3 3 3 7 7 ...
 $ target   : int  0 2 1 0 0 0 3 0 2 1 ...

A data.frame: 6 × 14
   age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
1    63   1   1    145   233     1     2    150     0     2.3     3     0     6     0
2    67   1   4    160   286     0     2    108     1     1.5     2     3     3     2
3    67   1   4    120   229     0     2    129     1     2.6     2     2     7     1
4    37   1   3    130   250     0     0    187     0     3.5     3     0     3     0
5    41   0   2    130   204     0     2    172     0     1.4     1     0     3     0
6    56   1   2    120   236     0     0    178     0     0.8     1     0     3     0
```

```
Injected negative BP at rows: 179 14 195 118 299
Injected NA BP at rows: 233 248 15 155 91
Injected extreme BP (>300) at rows: 94 265 204 303 141
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NAs
-138.0   120.0   130.0   131.3   140.0   397.0     5
```

	A data.frame: 15 × 2	
	trestbps	trestbps_cleaned
	<dbl>	<dbl>
	179	-130
	14	-120
	195	-120
	118	-138
	299	-110
	233	NA
	248	NA
	15	NA
	155	NA
	91	NA
	94	397
	265	376
	204	330
	303	311
	141	346

```
Mean of raw (dirty) trestbps: 131.2819
Mean of cleaned trestbps: 133.8908
Error while computing mean BP: All values are NA -- cannot compute a mean.

Mean of an all-NA vector: NA
```

```
Skipping ratio (chol=263, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=199, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=302, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=183, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=246, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=315, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=211, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=149, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=275, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=264, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=200, trestbps=0): Denominator is zero.
Skipping ratio (chol=NA, trestbps=80): Numerator or denominator is NA.
```

```
[1] NA 1.5 NA
      A data.frame: 6 × 3
```

	chol	trestbps_cleaned	chol_bp_ratio
	<dbl>	<dbl>	<dbl>
1	233	145	1.606897
2	286	160	1.787500
3	229	120	1.908333
4	250	130	1.923077
5	204	130	1.569231
6	236	120	1.966667

```
Loop-based system.time() (200 reps):
  user system elapsed
  0.013  0.000  0.014

Vectorized system.time() (200 reps):
  user system elapsed
  0.007  0.000  0.008
Unit: microseconds
      expr   min      lq    mean median      uq    max neval
loop_based 51.28 52.945 60.08109 55.485 60.740 108.33   100
vectorized 12.25 13.345 15.02018 14.010 15.095  61.82   100
```

```
Missing BP values (NA count): 10
Min BP: 94
Max BP: 250
Mean BP: 133.89
Median BP: 130

No negative values remain: TRUE
No values above 250 remain: TRUE

Validation PASSED: cleaned trestbps has no negative or >250 values.
```

2b:

```
'data.frame': 32563 obs. of 15 variables:
 $ age      : int  39 50 38 53 28 37 49 52 31 42 ...
 $ workclass : chr  "State-gov" "Self-emp-not-inc" "Private" "Private" ...
 $ fnlwgt   : int  77516 83311 215646 234721 338409 284582 160187 209642 45781 159449 ...
 $ education : chr  "Bachelors" "Bachelors" "HS-grad" "11th" ...
 $ education_num : int  13 13 9 7 13 14 5 9 14 13 ...
 $ marital_status : chr  "Never-married" "Married-civ-spouse" "Divorced" "Married-civ-spouse" ...
 $ occupation : chr  "Adm-clerical" "Exec-managerial" "Handlers-cleaners" "Handlers-cleaners" ...
 $ relationship : chr  "Not-in-family" "Husband" "Not-in-family" "Husband" ...
 $ race      : chr  "White" "White" "White" "Black" ...
 $ sex       : chr  "Male" "Male" "Male" "Male" ...
 $ capital_gain : int  2174 0 0 0 0 0 0 0 14084 5178 ...
 $ capital_loss : int  0 0 0 0 0 0 0 0 0 ...
 $ hours_per_week : int  40 13 40 40 40 40 16 45 50 40 ...
 $ native_country : chr  "United-States" "United-States" "United-States" "United-States" ...
 $ income     : chr  "<=50K" "<=50K" "<=50K" "<=50K" ...

A data frame: 6 x 15
  age workclass fnlwgt education education_num marital_status occupation relationship race sex capital_gain capital_loss hours_per_week native_country income
<int> <chr> <int> <chr> <int> <chr> <chr> <chr> <chr> <chr> <int> <int> <int> <chr> <chr>
1 39 State-gov 77516 Bachelors 13 Never-married Adm-clerical Not-in-family White Male 2174 0 40 United-States <=50K
2 50 Self-emp-not-inc 83311 Bachelors 13 Married-civ-spouse Exec-managerial Husband White Male 0 0 13 United-States <=50K
3 38 Private 215646 HS-grad 9 Divorced Handlers-cleaners Not-in-family White Male 0 0 40 United-States <=50K
4 53 Private 234721 11th 7 Married-civ-spouse Handlers-cleaners Husband Black Male 0 0 40 United-States <=50K
5 28 Private 338409 Bachelors 13 Married-civ-spouse Prof-specialty Wife Black Female 0 0 40 Cuba <=50K
6 37 Private 284582 Masters 14 Married-civ-spouse Exec-managerial Wife White Female 0 0 40 United-States <=50K
```

```
Injected negative BP at rows: 179 14 195 118 299
Injected NA BP at rows: 233 248 15 155 91
Injected extreme BP (>300) at rows: 94 265 204 303 141
  Min. 1st Qu. Median Mean 3rd Qu. Max. NAs
-138.0 120.0 130.0 131.3 140.0 397.0 5
```

A data.frame: 15 × 2

trestbps **trestbps_cleaned**

<dbl>

<dbl>

179 -130 NA

14 -120 NA

195 -120 NA

118 -138 NA

299 -110 NA

233 NA NA

248 NA NA

15 NA NA

155 NA NA

91 NA NA

94 397 250

265 376 250

204 330 250

303 311 250

141 346 250


```
Mean of raw (dirty) trestbps: 131.2819
Mean of cleaned trestbps: 133.8908
Error while computing mean BP: All values are NA -- cannot compute a mean.

Mean of an all-NA vector: NA
```

```
Skipping ratio (chol=263, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=199, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=302, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=183, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=246, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=315, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=211, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=149, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=275, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=264, trestbps=NA): Numerator or denominator is NA.
Skipping ratio (chol=200, trestbps=0): Denominator is zero.
Skipping ratio (chol=NA, trestbps=80): Numerator or denominator is NA.
```

```
[1] NA 1.5 NA
      A data.frame: 6 × 3
      chol trestbps_cleaned chol_bp_ratio
    <dbl>         <dbl>         <dbl>
1    233           145           1.606897
2    286           160           1.787500
3    229           120           1.908333
4    250           130           1.923077
5    204           130           1.569231
6    236           120           1.966667
```

	<pre>Loop-based system.time() (200 reps): user system elapsed 0.013 0.000 0.014 Vectorized system.time() (200 reps): user system elapsed 0.007 0.000 0.008 Unit: microseconds expr min lq mean median uq max neval loop_based 51.28 52.945 60.08109 55.485 60.740 108.33 100 vectorized 12.25 13.345 15.02018 14.010 15.095 61.82 100 Missing BP values (NA count): 10 Min BP: 94 Max BP: 250 Mean BP: 133.89 Median BP: 130 No negative values remain: TRUE No values above 250 remain: TRUE Validation PASSED: cleaned trestbps has no negative or >250 values.</pre>
Conclusion	Together, the two labs demonstrate that robust data cleaning in R depends on combining precise conditional logic, reusable functions, and proper exception handling with vectorized operations for efficiency - and that correctly distinguishing between NA, NaN, NULL, and "" is essential, since treating them interchangeably would silently produce incorrect or incomplete cleaning. The resulting pipelines produced two validated, analysis-ready datasets (cleaned_heart_data.csv and cleaned_adult_data.csv), confirming that a structured, checkable cleaning process is a prerequisite for reliable downstream statistical or machine learning work.